X Gate: Mathematical Proof & Complexity Analysis

What the X Gate Does to Stabilizers

The Pauli X gate acts on stabilizer tableaux via **conjugation**: for any stabilizer generator g, the new stabilizer after applying X_q is $X_q \cdot g \cdot X_q^\dagger = X_q \cdot g \cdot X_q$ (since $X = X^\dagger$).

We need to know how X_q conjugates each single-qubit Pauli on qubit q:

$$X_q \cdot I_q \cdot X_q = I_q$$ $$X_q \cdot X_q \cdot X_q = X_q$$ $$X_q \cdot Y_q \cdot X_q = X(iXZ)X = i \cdot X^2ZX = i \cdot ZX = -iXZ \cdot X = -Y_q$$ $$X_q \cdot Z_q \cdot X_q = -Z_q$$

So the sign rule is:

> **X_q conjugation flips the sign of any Pauli row that contains Z or Y on qubit q.**

Why? The Formal Proof

A stabilizer row P acting on n qubits is a tensor product. Only the component on qubit q matters for the sign update. That component is one of ${I, X, Y, Z}$, encoded as $(x_q, z_q) \in {(0,0),(1,0),(1,1),(0,1)}$.

Claim: $X_q P X_q = (-1)^{z_q} P$

Proof by exhaustive case analysis on the qubit-q component:

(x_q, z_q)	Pauli	$X P X$ on qubit q	Sign flip?
$(0,0)$	I	$X I X = I$	No — $z_q = 0$ ✓
$(1,0)$	X	$X X X = X$	No — $z_q = 0$ ✓
$(0,1)$	Z	$X Z X = -Z$	Yes — $z_q = 1$ ✓
$(1,1)$	$Y = iXZ$	$X(iXZ)X = i(X^2)(ZX) = i \cdot ZX = -iXZ = -Y$	Yes — $z_q = 1$ ✓

The key identity for the Y case spelled out: $$XYX = X(iXZ)X = i \underbrace{(XX)}_{=I} ZX = iZX = i(-XZ) = -iXZ = -Y$$

using $ZX = -XZ$ (anticommutativity). $\square$

The sign flip rule is purely: **if $z_q = 1$, flip $r_phase[r]$**. The x_q, z_q bits themselves are unchanged, because conjugation by X doesn't change what *type* of Pauli it is — only its sign.

Your Implementation

```
def x(self, q: int) -> None:
    for r in range(2 * self.n):
        if self.z_mat[r][q] == 1:
            self.r_phase[r] ^= 1
```

This is a direct encoding of the proof: iterate all $2n$ rows (destabilizers + stabilizers), and for each row where $z_mat[r][q] = 1$ (meaning the Pauli on qubit q is Z or Y), flip the phase bit. No x_mat modification needed.

Time Complexity

Your Implementation

Step	Operations	Cost
Iterate all rows	$2n$ iterations	$O(n)$
Per row: read `z_mat[r][q]`	1 read	$O(1)$
Per row: conditional XOR on `r_phase[r]`	1 write (branch)	$O(1)$
Total		**$O(n)$**

Memory access pattern: a single column scan over `z_mat`, which is a list-of-lists in your Python implementation. Each `z_mat[r][q]` access is a Python list index — $O(1)$ but with high constant factor due to interpreter overhead.

CHP's Implementation (Aaronson–Gottesman 2004)

CHP stores the tableau as packed bitfields — each row is a C `unsigned long` array where bits are packed. The X gate in CHP is:

```
for (r = 0; r < 2*n; r++)
    if (z[r][qubit])
        phase[r] = !phase[r];
```

Structurally identical, so also $O(n)$ in the number of row iterations, but with important constant-factor differences:

Factor	Your Python	CHP (C)
Row iteration	Python `for` loop	C `for` loop
Memory layout	List of Python `int` objects, pointer-indirect	Contiguous `int` array, cache-friendly
Per-element access	PyObject deref + bounds check	Direct memory read
Phase flip	Python XOR on boxed int	Bitwise NOT on C int
Constant factor (approx)	~50–100× slower	Baseline

CHP also optionally uses **packed bitfields** for larger simulations: entire rows are packed into `unsigned long` words, so a row multiplication becomes $O(n/w)$ with word size $w = 64$. For the X gate specifically, packing doesn't help since you're accessing a single *column* (qubit q across all rows) rather than full rows — you'd still need $2n$ bit extractions.

Comparison Summary

Metric	Your Code	CHP
Algorithmic complexity	$O(n)$	$O(n)$
Column access pattern	Cache-unfriendly (row-major list-of-lists)	Cache-unfriendly (same, but C arrays)
Practical speed at $n=1000$	~microseconds (CPython)	~nanoseconds
Correctness vs CHP	✓ Identical logic	Reference

The algorithm is optimal — you cannot do better than $O(n)$ since you must in principle inspect every row to check whether conjugation affects its phase. The gap between your implementation and CHP is purely a Python-vs-C constant factor, not algorithmic. If you wanted to close this gap, the path would be NumPy column masking: `self.r_phase ^= self.z_mat[:, q]` as arrays, which would recover roughly the same constant factor as CHP via vectorized C under the hood.